Unidad 3 · Ejercicios 2 y 3

WPF · XAML · Repaso rápido de examen

EJERCICIO 02 · Ribbon · Cinta de opciones Fluent

Fluent:Ribbon · RibbonTabItem · RibbonGroupBox · Backstage

CONCEPTO CLAVE Modelar navegación de estilo Office con pestañas, grupos de comandos y una vista Backstage para opciones generales.

SNIPPET DE INTERFAZ · XAML

```
<mah:MetroWindow x:Class="di.unidad3.ejercicio2.MainWindow"
  xmlns:Fluent="urn:fluent-ribbon"
  xmlns:mah="http://metro.mahapps.com/winfx/xaml/controls">
  <Fluent:Ribbon>
    <Fluent:Ribbon.Menu><Fluent:Backstage>
      <Fluent:BackstageTabControl>
        <Fluent:BackstageTabItem Header="Opciones" />
        <Fluent:Button Header="Cerrar" />
      </Fluent:BackstageTabControl>
    </Fluent:Backstage></Fluent:Ribbon.Menu>
    <Fluent:RibbonTabItem Header="Prestamos">
      <Fluent:RibbonGroupBox Header="Operaciones">
        <Fluent:Button Header="Salida" />
        <Fluent:Button Header="Devolución" />
      </Fluent:RibbonGroupBox>
    </Fluent:RibbonTabItem>
  </Fluent:Ribbon>
</mah:MetroWindow>
```

SNIPPET DE LÓGICA · C#

```
public partial class MainWindow : MetroWindow
{
    public MainWindow() { InitializeComponent(); }
}
// Las acciones de Ribbon no tienen handlers aquí.
```

POSIBLES TRAMPAS DE EXAMEN

- **Namespace:** `xmlns:Fluent="urn:fluent-ribbon"` identifica los controles de Fluent Ribbon.
- **Jerarquía:** Ribbon contiene pestañas; cada `RibbonTabItem` organiza `RibbonGroupBox`.
- **Backstage no es un RibbonTab:** se declara dentro de `Ribbon.Menu`.
- **Prototipo visual:** los botones del Ribbon/Backstage no están enlazados a lógica en el code-behind.

Unidad 3 · Ejercicios 2 y 3

WPF · XAML · Repaso rápido de examen

EJERCICIO 03 · Menú lateral · Expander y ListView

DockPanel · Panel izquierdo · Acordeón Material Design · Binding SubItems

CONCEPTO CLAVE Construir una navegación lateral con secciones expandibles y una zona central independiente para el contenido.

SNIPPET DE INTERFAZ · XAML

```
<DockPanel>
  <Grid DockPanel.Dock="Left" Width="250" Background="#E5303030">
    <StackPanel Margin="10">
      <Expander IsExpanded="False" Header="Settings"
        Style="{StaticResource MaterialDesignExpander}">
        <ListView ItemsSource="{Binding Path=SubItems}"
          Style="{StaticResource MaterialDesignListView}">
          <ListViewItem Content="Users" />
          <ListViewItem Content="Hardware" />
          <ListViewItem Content="Software" />
        </ListView>
      </Expander>
    </StackPanel>
  </Grid>
  <Grid x:Name="gridCentral" />
</DockPanel>
```

SNIPPET DE LÓGICA · C#

```
public partial class MainWindow : MetroWindow
{
    public MainWindow() { InitializeComponent(); }
}
// No se asigna DataContext en este code-behind.
```

POSIBLES TRAMPAS DE EXAMEN

- **ItemsSource + Items:** este XAML asigna `ItemsSource` y además declara `ListViewItem` hijos. En WPF no se deben mezclar ambas formas.
- **Binding sin origen:** `SubItems` requiere un DataContext que lo exponga; el code-behind mostrado no lo asigna.
- **Expander cerrado:** `IsExpanded="False"` inicia contraído; las opciones no se ven hasta abrirlo.
- **Recursos:** los estilos `MaterialDesignExpander/ListView` requieren los diccionarios correspondientes en App.xaml.